
```

#!/usr/bin/env python3

# Run me like this:
# $ python3 padding_oracle.py
# "https://project1.eecs388.org/username/paddingoracle/verify" "5a7793d3..."
# or select "Padding Oracle" from the VS Code debugger

import json
import sys
import time
from typing import Dict, List

import requests

# Create one session for each oracle request to share. This allows the
# underlying connection to be re-used, which speeds up subsequent requests!
s = requests.session()

def oracle(url: str, messages: List[bytes]) -> List[Dict[str, str]]:
    while True:
        try:
            r = s.post(url, data={"message": [m.hex() for m in messages]})
            r.raise_for_status()
            return r.json()
        # Under heavy server load, your request might time out. If this happens,
        # the function will automatically retry in 10 seconds for you.
        except requests.exceptions.RequestException as e:
            sys.stderr.write(str(e))
            sys.stderr.write("\nRetrying in 10 seconds...\n")
            time.sleep(10)
            continue
        except json.JSONDecodeError as e:
            sys.stderr.write("It's possible that the oracle server is overloaded
right now, or that provided URL is wrong.\n")
            sys.stderr.write("If this keeps happening, check the URL. Perhaps your
username is not set.\n")
            sys.stderr.write("Retrying in 10 seconds...\n\n")
            time.sleep(10)
            continue

def decrypt_block(oracle_url, iv, block):
    block_size = 16
    decrypted = bytearray(block_size)
    intermediate = bytearray(block_size)

    for i in range(1, block_size + 1):
        for guess in range(256):
            modified_block = bytearray(iv[:-i] + bytes([guess ^ (i ^
intermediate[block_size-i])]) + decrypted[-i+1:])
            if oracle(oracle_url, [modified_block + block])[0]["status"] ==
"valid":
                decrypted[block_size-i] = guess ^ i
                intermediate[block_size-i] = guess
                break

    return bytes(decrypted)

def main():

```

```

if len(sys.argv) != 3:
    print(f"usage: {sys.argv[0]} ORACLE_URL CIPHERTEXT_HEX", file=sys.stderr)
    sys.exit(-1)
oracle_url, message = sys.argv[1], bytes.fromhex(sys.argv[2])
ciphertext = message
if oracle(oracle_url, [message])[0]["status"] != "valid":
    print("Message invalid", file=sys.stderr)

block_size = 16
if len(ciphertext) % block_size != 0:
    print("Ciphertext length is not a multiple of the block size.",
file=sys.stderr)
    return

# Split the ciphertext into blocks
blocks = [ciphertext[i:i+block_size] for i in range(0, len(ciphertext),
block_size)]
decrypted_message = b''

# Implement the decryption logic here
for block_index in range(len(blocks) - 1, 0, -1):
    decrypted_block = decrypt_block(oracle_url, blocks[block_index - 1],
blocks[block_index])
    decrypted_message = decrypted_block + decrypted_message

# Assuming the IV is the first block and not required for decryption
# Remove PKCS#7 padding from the final plaintext block
last_block_padding = decrypted_message[-1]
decrypted_message = decrypted_message[:-last_block_padding]

print(decrypted_message.decode('utf-8'))

#
# TODO: Decrypt the message
#

# decrypted = "TODO"
# print(decrypted)

if __name__ == '__main__':
    main()

```